

Laboratorio 1: Vulnerabilidad de inyección SQL en la cláusula WHERE que permite la recuperación de datos ocultos

Nivel: **Apprentice**

Tipo de SQL Injection: inyección SQL en el filtro de categoría de la base de datos de producto.

1. Objetivo del laboratorio:

2. Explicación de la vulnerabilidad:

3. Identificación de la vulnerabilidad

4. Herramientas utilizadas:

- **Burp Suite**
- **Navegador web**
- **Plataforma PortSwigger Web Security Academy**

Explicar brevemente el uso de Burp:

Burp Suite se utiliza para interceptar las peticiones HTTP enviadas al servidor, permitiendo modificar los parámetros y probar diferentes payloads de SQL Injection.

5. Procedimiento paso a paso

- I. **Acceder al laboratorio de la plataforma PortSwigger.**
- II. **En la ventana que se abrió, se mostró información de productos con filtro**
- III. **Se usó Burp Suite, en la categoría Proxy para interceptar las peticiones HTTP.**
- IV. **Se consultaron algunas categorías de productos en la página para generar solicitudes HTTP.**
- V. **Burp Suite capturó las consultas enviadas al servidor.**
- VI. **Se pudo identificar la vulnerabilidad dentro de la categoría, con las solicitudes enviadas.**
- VII. **Se modificó el parámetro, sustituyéndolo por un payload usado para SQL Injection.**
- VIII. **Se mandó la petición al servidor.**
- IX. **La página mostró productos que anteriormente estaban ocultos, de esta forma se pudo confirmar la explotación de la vulnerabilidad.**
- X. **De esta forma, se pudo resolver el laboratorio.**

6. Payload utilizado

'+OR+1=1--

7. Explicación del payload

Es utilizado en ataques de inyección SQL para manipular consultas de bases de datos. Su propósito es forzar que una condición siempre sea verdadera ($1=1$), lo que puede permitir el acceso no autorizado o la exfiltración de datos.

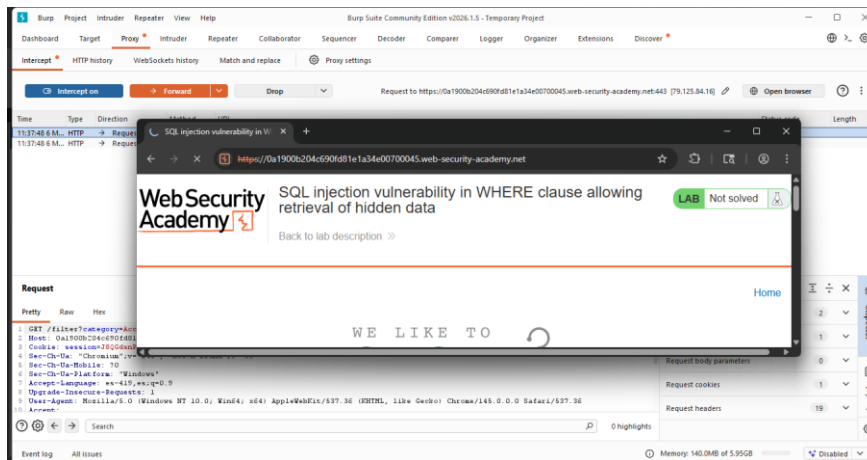
8. Resultado del ataque

Después de haber modificado el parámetro con el payload, se pudo observar productos que deberían de estar ocultos, ya que no pertenecían a cierta categoría.

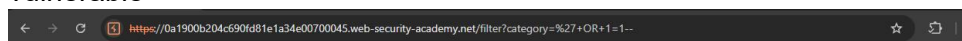
9. Evidencias

Capturas de pantalla

- Request interceptado en Burp

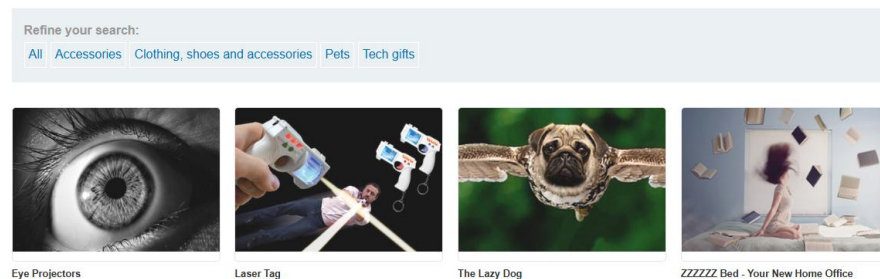


- Respuesta vulnerable

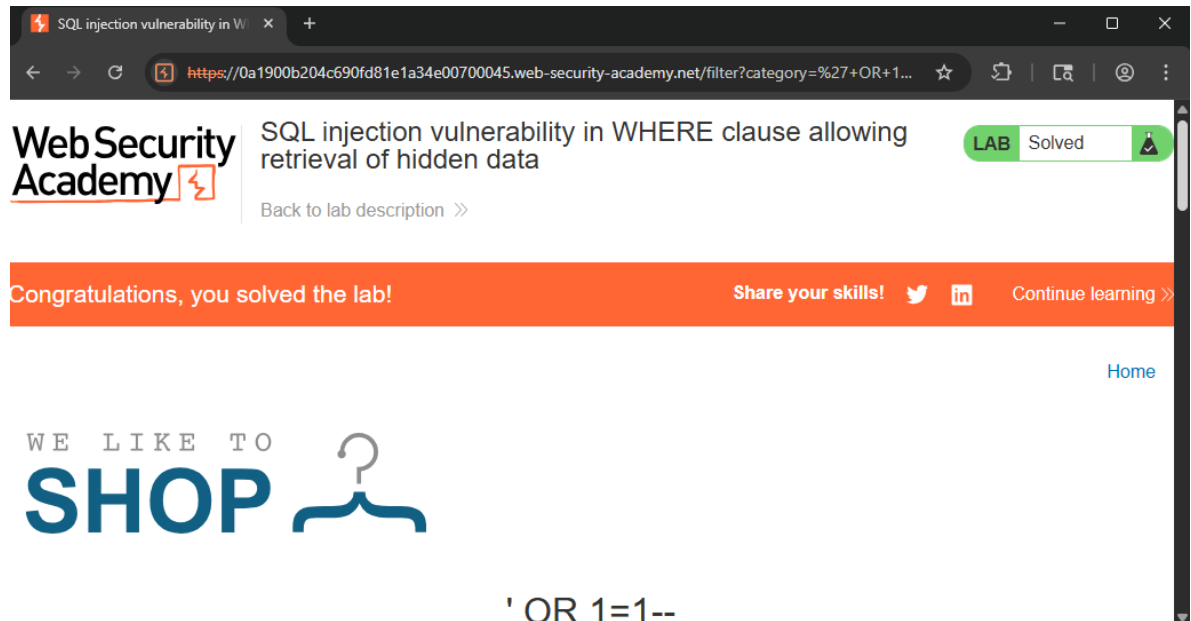


WE LIKE TO
SHOP 

' OR 1=1--



- Pantalla de **Lab Solved**



10. Impacto de la vulnerabilidad

- I. **Acceso no autorizado:** Permite hacer un bypass en la autenticación.
- II. **Exfiltración de datos:** Acceso a toda la base de datos (usuarios, contraseñas, información sensible).
- III. **Modificación o eliminación de datos:** Si la consulta permite UPDATE o DELETE.
- IV. **Elevación de privilegios:** Acceder como administrador o usuario con más permisos.
- V. **Denegación de servicio (DoS):** Bloquear o corromper la base de datos.

11. Mitigación

Para evitar vulnerabilidades como la inyección SQL (incluyendo exploits como OR 1=1--):

1. **Usar consultas parametrizadas o sentencias preparada:** Jamás concatenar directamente entradas de usuario en consultas SQL.
2. **Validar y sanitizar entradas:** Filtrado de caracteres especiales en datos de usuarios.
3. **Aplicar el principio de mínimo privilegio** — Las cuentas de base de datos deben tener solo los permisos necesarios.

4. **Usar ORM (Mapeo Objeto-Relacional) o frameworks seguros** — Herramientas como SQLAlchemy, Hibernate o Django ORM ayudan a prevenir inyecciones automáticamente.
5. **Realizar pruebas de seguridad** — Escaneos de vulnerabilidades y pentesting regulares.